Code Sample*

UNDER-10MB COPY

Anonymous

This document contains excerpts from the replication code of a paper titled "Inequality in a Potential Equalizer of Opportunity: Variation in the Accessibility and Net Benefits of English-Medium Instruction in India," with figures selectively rasterized to reduce file size. Excerpts are followed by their corresponding outputs when appropriate. The project may have been updated since this file was generated; the most up-to-date versions of this document, the full paper, and its replication package are available online, and I would be happy to provide links for those interested. This PDF can be generated by running `make samples` or `scripts/run_full_build.sh` from the repo root.

Historical Geography and Measurement

Reconstructing District Boundary Transfers Between 1991 and 2001

Kumar and Somanathan's (2016) district concordance reports how the population of each 1991 district was distributed across 2001 districts after boundary changes. Some district names wrap onto a second line in the original table, leaving the 1991 population blank on that line. I inspect those rows before filling down repeated values. If a wrapped line contains transfer percentages, I retain it as a separate allocation record and restore its 1991 district name and population. If it contains only text, I join that text to the district name on the preceding line and remove the extra row. After reconstructing the table, I verify that the shares of each 1991 district's population assigned to 2001 districts sum to 100%, allowing 0.05 percentage points for rounding.

File: `R/io/read_inputs.R`

*GPT models 5.5 and 5.6 were used for troubleshooting, reviewing results, and generating certain portions of this code from my detailed outlines and pseudocode, after which I reviewed and edited their responses extensively. All ideas, analytical work, and errors are my own.

```r
# Repair physical row wraps in the historical district transfer table
#
# A blank 1991 population field marks a candidate continuation row. Transfer
# fields distinguish a genuine allocation record from a row that only continues
# a printed name. For split source names, lowercase continuations remove a
# hyphen inserted by a line break; uppercase continuations preserve a hyphen
# that belongs to the district name.
repair_district_carveout_wrapped_rows <- function(x) {
  out <- safe_df(x)
  if (!nrow(out)) return(out)
  blank <- function(value) is.na(value) || !nzchar(trimws(as.character(value)))
  drop <- rep(FALSE, nrow(out))
  for (i in seq_len(nrow(out))) {
    if (i == 1L || !blank(out$pop_1991[[i]])) next
    previous <- i - 1L
    source_continuation <- !blank(out$district_1991[[i]])
    target_continuation <- !blank(out$district_2001[[i]])
    has_transfer <- !blank(out$pct_01in91[[i]]) || !blank(out$pct_91in01[[i]])

    if (source_continuation) {
      previous_name <- plain_chr(out$district_1991[[previous]])
      if (!length(previous_name) || !grepl("-$", trimws(previous_name[[1L]]))) {
        stop("Unexpected wrapped row in the district boundary table.", call. = FALSE)
      }
      continuation <- plain_chr(out$district_1991[[i]])[[1L]]
      combined <- join_district_carveout_wrapped_label(previous_name[[1L]], continuation)
      out$district_1991[[previous]] <- combined
      if (has_transfer) {
        out$district_1991[[i]] <- combined
        out$pop_1991[[i]] <- out$pop_1991[[previous]]
      }
    }

    if (!has_transfer && (source_continuation || target_continuation)) {
      if (target_continuation) {
        if (blank(out$district_2001[[previous]])) {
          stop("Unexpected continuation of a 2001 district name in the district boundary table.", call. =
            ↪ FALSE)
        }
        previous_target <- trimws(as.character(out$district_2001[[previous]]))
```

```
        continuation <- trimws(as.character(out$district_2001[[i]]))
        out$district_2001[[previous]] <- join_district_carveout_wrapped_label(
          previous_target, continuation
        )
      }
      drop[[i]] <- TRUE
    }
  }
  out[!drop, , drop = FALSE]
}

district_carveout_rounding_tolerance_pp <- function() 0.05

# Validate population shares in the 1991 to 2001 district transfer table
#
validate_district_carveout_shares <- function(
    x,
    rounding_tolerance_pp = district_carveout_rounding_tolerance_pp()) {
  out <- safe_df(x)
  if (!nrow(out)) return(invisible(TRUE))
  share_cols <- c("pct_01in91", "pct_91in01")
  for (field in share_cols) {
    values <- num(out[[field]])
    bad <- is.finite(values) & (values < 0 | values > 100)
    if (any(bad)) {
      stop("District transfer shares must lie in [0, 100].", call. = FALSE)
    }
  }
  source_share <- num(out$pct_01in91)
  if (any(!is.finite(source_share))) {
    stop("Shares of each 1991 district assigned to 2001 districts must be complete.", call. = FALSE)
  }
  # Group transfers by the 1991 district name and population printed in the
  # table.
  source_key <- paste(out$district_1991, out$pop_1991, sep = "__")
  source_sum <- vapply(split(source_share, source_key), sum, numeric(1))
  if (any(abs(source_sum - 100) > rounding_tolerance_pp)) {
    stop("Shares of each 1991 district assigned to 2001 districts must sum to 100 within rounding
    ↪  tolerance.", call. = FALSE)
  }
  invisible(TRUE)
```

```
}


read_district_carveouts <- function(path) {
  out <- utils::read.csv(
    path,
    header = FALSE,
    col.names = c("district_1991", "pop_1991", "district_2001", "pct_01in91", "pct_91in01"),
    stringsAsFactors = FALSE,
    na.strings = c("", "NA"),
    check.names = FALSE
  )
  # Repair layout blanks before filling downward because the blanks themselves
  # identify candidate continuation rows.
  out <- repair_district_carveout_wrapped_rows(out)
  out$district_1991 <- fill_down_missing(out$district_1991)
  out$pop_1991 <- num(gsub(",", "", fill_down_missing(out$pop_1991), fixed = TRUE))
  out$pct_01in91 <- num(out$pct_01in91)
  out$pct_91in01 <- num(out$pct_91in01)
  validate_district_carveout_shares(out)
  out
}
```

Generating District Match Candidates for Review

District names vary across survey waves and reference years. For each survey district, I compare only reference districts in the same state. I compute Jaro–Winkler, normalized Damerau–Levenshtein, trigram cosine, and token Jaccard similarities for every candidate pair and average the four scores. I prefer 2001 reference districts for the 2007–08 NSS and 2011 reference districts for later waves. I flag a pair for review only when the survey district and reference district rank each other first and their directional terms agree. I also require Jaro–Winkler similarity of at least 0.90, normalized Damerau–Levenshtein similarity of at least 0.70, trigram similarity of at least 0.55, and a margin of at least 0.05 over the next candidate name. Token Jaccard enters the average score without an additional threshold. I retain the highest-ranked candidates for subsequent district lineage review.

File: `R/districts/lineage_matching.R`

```
score_match_candidates <- function(source_roster, reference_units, excluded_source_ids = character()) {
  need_pkg("stringdist", "district-lineage candidate scores")
  source_roster <- safe_df(source_roster)
  reference_units <- safe_df(reference_units)
```

```r
if (!"raw_district" %in% names(source_roster)) source_roster$raw_district <- source_roster$district_std
source_open <- source_roster[!source_roster$source_row_id %in% excluded_source_ids, , drop = FALSE]
if (!nrow(source_open)) return(empty_source_candidates())

pairs <- merge(
  source_open, reference_units,
  by = "state_std", all = FALSE, sort = FALSE,
  suffixes = c("_source", "_candidate")
)
if (!nrow(pairs)) return(empty_source_candidates())
source_name <- pairs$district_std_source
candidate_name <- pairs$district_std_candidate
# p = 0.1 applies Winkler's prefix adjustment rather than plain Jaro distance.
pairs$jw <- 1 - stringdist::stringdist(
  source_name, candidate_name, method = "jw", p = 0.1
)
dl_distance <- stringdist::stringdist(source_name, candidate_name, method = "dl")
denom <- pmax(nchar(source_name), nchar(candidate_name), 1L)
pairs$dl <- pmax(0, 1 - dl_distance / denom)
pairs$trigram <- stringdist::stringsim(source_name, candidate_name, method = "cosine", q = 3)
pairs$token <- token_jaccard_similarity(source_name, candidate_name)
pairs$score <- rowMeans(pairs[c("jw", "dl", "trigram", "token")], na.rm = TRUE)

ranked <- safe_bind_rows(lapply(split(seq_len(nrow(pairs)), pairs$source_row_id), function(i) {
  x <- pairs[i, , drop = FALSE]
  # Prefer the reference year closest to the survey wave.
  preference <- vintage_preference(x$wave[[1]])
  vintage_rank <- match(x$reference_vintage, preference, nomatch = length(preference) + 1L)
  x <- x[order(-x$score, -x$jw, vintage_rank, x$unit_id), , drop = FALSE]
  x$rank <- match(x$district_std_candidate, unique(x$district_std_candidate))
  distinct_scores <- x$score[!duplicated(x$district_std_candidate)]
  second <- if (length(distinct_scores) >= 2L) distinct_scores[[2]] else -Inf
  x$margin <- ifelse(x$rank == 1L, x$score - second, NA_real_)
  x$preferred_vintage <- !duplicated(x$district_std_candidate)
  x
}))

# Require the survey district and reference district to rank each other first.
candidate_groups <- split(seq_len(nrow(ranked)), ranked$unit_id)
best_by_candidate <- stats::setNames(
  vapply(candidate_groups, function(i) {
```

```
      score <- ranked$score[i]
      ranked$source_row_id[i[which.max(score)]]
    }, character(1)),
    names(candidate_groups)
  )
  ranked$reciprocal_nearest <- ranked$rank == 1L &
    unname(best_by_candidate[ranked$unit_id]) == ranked$source_row_id
  threshold <- district_match_candidate_thresholds()
  # This flag prioritizes the pair for the subsequent lineage review.
  ranked$high_precision_candidate <- ranked$rank == 1L & ranked$preferred_vintage &
    ranked$reciprocal_nearest &
    directional_tokens_compatible(ranked$district_std_source, ranked$district_std_candidate) &
    ranked$jw >= threshold[["jw"]] & ranked$dl >= threshold[["dl"]] &
    ranked$trigram >= threshold[["trigram"]] & ranked$margin >= threshold[["margin"]]
  ranked <- ranked[ranked$rank <= 5L, , drop = FALSE]

  data.frame(
    source_row_id = ranked$source_row_id,
    wave = ranked$wave,
    source_code = ranked$source_code,
    state_std = ranked$state_std,
    source_name_raw = ranked$raw_district,
    source_name = ranked$district_std_source,
    candidate_unit = ranked$unit_id,
    candidate_name = ranked$district_std_candidate,
    candidate_source_id = ranked$source_id,
    reference_vintage = ranked$reference_vintage,
    candidate_method = "fuzzy_name_candidate",
    rank = ranked$rank,
    jw = ranked$jw,
    dl = ranked$dl,
    trigram = ranked$trigram,
    token = ranked$token,
    score = ranked$score,
    margin = ranked$margin,
    reciprocal_nearest = ranked$reciprocal_nearest,
    high_precision_candidate = ranked$high_precision_candidate,
    stringsAsFactors = FALSE
  )
}
```

Reconstructing District Data from the 1991 Language Atlas

In the 1991 Census Language Atlas, each district appears on eight different pages of language data. Each district's printed identifier combines its Census state code with a serial number within the state, but some repeated copies omit the serial or contain text extraction errors. When the current row has no serial, I use the repeated copies only if the exact same printed row label within the same Atlas block is associated with one serial number throughout. I then verify that the state headings cover the complete reviewed set of 1991 states and agree with the state codes in the Primary Census Abstract. I accept a reconstructed district only when its state and district codes identify one Primary Census Abstract district and the Atlas population is within 1% of that district's Census population.

File: `scripts/build_language_atlas_1991.py`

```
def cross_page_serial_evidence(cells: list[dict[str, object]]) -> dict[tuple[int, str], tuple[int, ...]]:
    """Collect district serials printed with repeated Language Atlas row labels.

    The Atlas prints each district on eight language pages. The district serial is
    the within-state number paired with the Census state code. If one copy omits
    that number, repeated copies can supply it only when their printed serials agree.
    """
    observed: dict[tuple[int, str], set[int]] = {}
    unique_rows = {
        (int(cell["block"]), int(cell["page"]), int(cell["row_sequence"]),
         str(cell["row_label_raw"]).strip(), str(cell["serial_candidate"]).strip())
        for cell in cells
    }
    for block, _page, _sequence, label, serial_raw in unique_rows:
        if not label or label == "INDIA":
            continue
        serial = int(serial_raw) if serial_raw else parse_leading_serial(label)
        if serial is not None:
            observed.setdefault((block, label), set()).add(serial)
    return {key: tuple(sorted(values)) for key, values in observed.items()}


def reconcile_district_serial(
    cell: dict[str, object],
    evidence: dict[tuple[int, str], tuple[int, ...]],
) -> tuple[int | None, str, str]:
    """Choose a district serial from the row itself or agreeing repeated copies."""
```

```
    label = str(cell["row_label_raw"]).strip()
    serial_column = int(cell["serial_candidate"]) if str(cell["serial_candidate"]).strip() else None
    serial_label = parse_leading_serial(label)
    repeated = evidence.get((int(cell["block"]), label), ())
    repeated_text = ";".join(map(str, repeated))

    if serial_column is not None and serial_label is not None and serial_column != serial_label:
        return None, "direct_conflict", repeated_text
    direct = serial_column if serial_column is not None else serial_label
    direct_source = "serial_column" if serial_column is not None else "label_prefix"

    if direct is not None:
        # Prefer a serial printed on the current row. Use repeated copies of the
        # same label only when the current row has no serial because
        # continuation pages contain occasional text extraction errors.
        return direct, direct_source, repeated_text
    if len(repeated) == 1:
        return repeated[0], "cross_page_exact_label", repeated_text
    if len(repeated) > 1:
        return None, "cross_page_conflict", repeated_text
    return None, "missing", repeated_text


def build_district_population_validation(
    cells: list[dict[str, object]],
    state_crosswalk: dict[str, dict[str, str]],
    pca91: dict[tuple[str, str], int],
) -> list[dict[str, object]]:
    rows = merge_population_continuations(cells, state_crosswalk)
    serial_evidence = cross_page_serial_evidence(cells)
    seen_state_codes: set[str] = set()
    current_state: dict[str, str] | None = None
    candidates: list[dict[str, object]] = []
    for cell in rows:
        label = str(cell["row_label_raw"]).strip()
        if label == "INDIA":
            continue
        mapped_state = state_crosswalk.get(label)
        if mapped_state is not None and mapped_state["state_code_1991"] not in seen_state_codes:
            current_state = mapped_state
            seen_state_codes.add(mapped_state["state_code_1991"])
```

```
        continue

    serial, serial_source, cross_page_serials = reconcile_district_serial(cell, serial_evidence)

    state_code = current_state["state_code_1991"] if current_state else ""
    state_name = current_state["state_name_1991"] if current_state else ""
    district_code = f"{serial:02d}" if serial is not None else ""
    pca_population = pca91.get((state_code, district_code)) if state_code and district_code else None
    # The extractor stores any parsed numeric cell in one candidate field;
    # on the population page that field is the district population.
    atlas_population = (
        int(cell["speaker_count_candidate"])
        if str(cell["speaker_count_candidate"]).strip()
        else None
    )
    if atlas_population is not None and pca_population is not None and pca_population > 0:
        abs_diff = abs(atlas_population - pca_population)
        rel_diff = abs_diff / pca_population
    else:
        abs_diff = None
        rel_diff = None
    candidates.append({
        "block": cell["block"],
        "page": cell["page"],
        "row_sequence": cell["row_sequence"],
        "row_sequence_end": cell["row_sequence_end"],
        "row_merge_status": cell["row_merge_status"],
        "row_label_raw": label,
        "state_code_1991": state_code,
        "state_name_1991": state_name,
        "district_code_1991": district_code,
        "serial_source": serial_source,
        "cross_page_serials": cross_page_serials,
        "atlas_population_raw": cell["raw_value"],
        "atlas_population_candidate": "" if atlas_population is None else atlas_population,
        "pca91_population": "" if pca_population is None else pca_population,
        "population_abs_diff": "" if abs_diff is None else abs_diff,
        "population_relative_diff": "" if rel_diff is None else round(rel_diff, 8),
        "identity_status": "",
        "population_status": "",
        "promotion_status": "review_required",
```

```
    })

expected_state_codes = {row["state_code_1991"] for row in state_crosswalk.values()}
if seen_state_codes != expected_state_codes:
    missing = sorted(expected_state_codes - seen_state_codes)
    extra = sorted(seen_state_codes - expected_state_codes)
    raise ValueError(
        "Language Atlas state headings do not match the expected 1991 states; "
        f"missing={missing}; extra={extra}"
    )
pca_state_codes = {state_code for state_code, _ in pca91}
if pca_state_codes != expected_state_codes:
    raise ValueError("Language Atlas state names and 1991 Primary Census Abstract state codes disagree")

key_counts: dict[tuple[str, str], int] = {}
for row in candidates:
    key = (str(row["state_code_1991"]), str(row["district_code_1991"]))
    if all(key):
        key_counts[key] = key_counts.get(key, 0) + 1

for row in candidates:
    key = (str(row["state_code_1991"]), str(row["district_code_1991"]))
    if not row["state_code_1991"]:
        identity_status = "missing_state_context"
    elif row["serial_source"] in {"direct_conflict", "cross_page_conflict"}:
        identity_status = "serial_conflict"
    elif not row["district_code_1991"]:
        identity_status = "missing_district_serial"
    elif key_counts.get(key, 0) > 1:
        identity_status = "duplicate_district_code"
    elif key not in pca91:
        identity_status = "missing_pca_district"
    else:
        identity_status = "candidate_official_code"
    row["identity_status"] = identity_status

    if row["atlas_population_candidate"] == "":
        population_status = "atlas_population_unparsed"
    elif row["pca91_population"] == "":
        population_status = "pca_population_missing"
    elif float(row["population_relative_diff"]) <= POPULATION_RELATIVE_TOLERANCE:
```

```
            population_status = "within_1pct"
        else:
            population_status = "over_1pct"
        row["population_status"] = population_status
        if identity_status == "candidate_official_code" and population_status == "within_1pct":
            row["promotion_status"] = "population_validated_candidate"
    return candidates
```

Bounding Historical Linguistic Distance with Unresolved Population Counts

After reconciling language names, some district population remains attached to language entries without an accepted Shastry distance. For each district, I first calculate the speaker-weighted mean among accepted languages with a positive distance. Hindi and other zero-distance counts, along with native English counts, are already resolved and do not enter that mean. I then calculate how far the mean could vary if all unresolved population were assigned the smallest or largest accepted positive Shastry degree. I also include the observed mean because unresolved counts can belong to groups that do not enter the positive-distance denominator. I use the preferred historical measure only when accepted counts cover at least 99% of the district population and the resulting interval is no wider than 0.5 Shastry degree.

File: `R/measures/build_linguistic_distance_iv.R`

```
historical_linguistic_population_coherent <- function(speakers, population) {
  speakers <- num(speakers)
  if (!is.numeric(population) || length(population) != 1L ||
      !is.finite(population) || population <= 0) {
    stop("Historical linguistic bounds require a positive district population.", call. = FALSE)
  }
  valid_count <- is.finite(speakers) & speakers >= 0
  valid_total <- sum(speakers[valid_count], na.rm = TRUE)
  isTRUE(valid_total <= population)
}

historical_linguistic_distance_bounds <- function(
    speakers, degree, language, population, nonzero_degree_range = c(1, 5)) {
  speakers <- num(speakers)
  degree <- num(degree)
  language <- normalize_language_label(language)
  population_coherent <- historical_linguistic_population_coherent(speakers, population)
```

```
valid_count <- is.finite(speakers) & speakers >= 0
english <- valid_count & language == "English"
known_nonzero <- valid_count & is.finite(degree) & degree > 0 & !english
known_zero <- valid_count & is.finite(degree) & degree == 0 & !english
# These resolved speakers are excluded from the positive distance mean.
known_excluded <- known_zero | english

nonzero_speakers <- sum(speakers[known_nonzero], na.rm = TRUE)
numerator <- sum(speakers[known_nonzero] * degree[known_nonzero], na.rm = TRUE)
known_excluded_speakers <- sum(speakers[known_excluded], na.rm = TRUE)
resolved_speakers <- nonzero_speakers + known_excluded_speakers
unresolved_mass <- max(0, population - resolved_speakers)
point <- if (nonzero_speakers > 0) numerator / nonzero_speakers else NA_real_

nonzero_degree_range <- sort(unique(num(nonzero_degree_range)))
if (length(nonzero_degree_range) < 2L || any(!is.finite(nonzero_degree_range)) ||
    any(nonzero_degree_range <= 0)) {
  stop("Historical linguistic distance bounds require a finite positive degree range.", call. = FALSE)
}
degree_min <- min(nonzero_degree_range)
degree_max <- max(nonzero_degree_range)
if (!population_coherent) {
  point <- lower <- upper <- width <- unresolved_mass <- NA_real_
} else if (!is.finite(point) || !is.finite(degree_min) || !is.finite(degree_max)) {
  lower <- upper <- width <- NA_real_
} else if (unresolved_mass <= 0) {
  lower <- upper <- point
  width <- 0
} else {
  lower_if_nonzero <- (numerator + unresolved_mass * degree_min) /
    (nonzero_speakers + unresolved_mass)
  upper_if_nonzero <- (numerator + unresolved_mass * degree_max) /
    (nonzero_speakers + unresolved_mass)
  # Unresolved population may consist of Hindi speakers, English speakers, or
  # language counts without an accepted distance value; in those cases the
  # positive distance mean is unchanged.
  lower <- min(point, lower_if_nonzero)
  upper <- max(point, upper_if_nonzero)
  width <- upper - lower
}
```

```
  list(
    point = point,
    lower = lower,
    upper = upper,
    width = width,
    nonzero_speakers = nonzero_speakers,
    known_excluded_speakers = known_excluded_speakers,
    resolved_speakers = resolved_speakers,
    unresolved_mass_upper_bound = unresolved_mass,
    population_coherent = population_coherent,
    degree_min = degree_min,
    degree_max = degree_max
  )
}
```

Output: `outputs/figures/appendix/appendix_historical_language_persistence.pdf`

Rendering code: `R/output/appendix_data_construction_exhibits.R` and `R/output/save_figures.R`

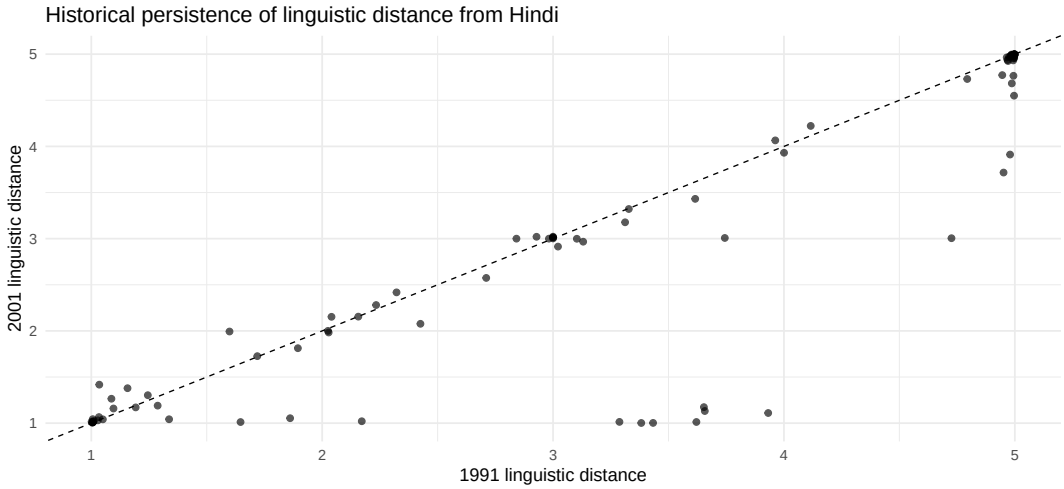


Figure 2: Persistence of speaker-weighted linguistic distance from Hindi from 1991 to 2001. Each point is a 2001 Census district with a reconstructed measure in both years; the dashed line is the 45-degree line. Sources: Census of India 1991 Language Atlas (Office of the Registrar General and Census Commissioner, India 2004) and Census of India 2001 language tables (Office of the Registrar General and Census Commissioner of India 2001).

Independent Schooling Measures and Survey Estimation

Reconstructing English-Medium Enrollment from Archived DISE Reports

I use District Information System for Education (DISE) district report cards from 2008–09 through 2014–15 to construct an independent measure of English-medium enrollment. For each reviewed page, I run Poppler's `pdftotext -layout` so that the extracted text retains the page spacing as far as possible. When languages appear in rows, I require a nearby `Total` header and read the final number on each English or Hindi row. When languages appear in columns, I locate the English and Hindi headings and read the nearest number on an explicit `Total` or `Grand Total` row. I reconstruct every recorded district and year from its reviewed page and check the resulting counts against the saved count table. If several archived reports cover the same district and year, the recorded report priority must select one report uniquely.

File: `scripts/build_dise_report_language_enrollment.py`

```
LANGUAGES = ("english", "hindi")
NUMBER = re.compile(r"(?<![\w.])(?:\d{1,3}(?:,\d{3})+|\d+)(?![\w.])")
MEDIUM_HEADING = re.compile(r"medium\s+of\s+instruction", re.IGNORECASE)
# Keep both searches local to the table region covered by the supported layouts.
MEDIUM_BLOCK_LINES = 28
TOTAL_HEADER_LOOKBACK = 6


def medium_block(text: str, radius: int = MEDIUM_BLOCK_LINES) -> list[str]:
    lines = text.splitlines()
    hit = next((i for i, line in enumerate(lines) if MEDIUM_HEADING.search(line)), None)
    if hit is None:
        raise ValueError("page has no 'medium of instruction' heading")
    return lines[hit : min(len(lines), hit + radius)]


def _has_total_header(lines: list[str], row_index: int) -> bool:
    start = max(0, row_index - TOTAL_HEADER_LOOKBACK)
    return any(re.search(r"\btotal\b", line, re.IGNORECASE) for line in lines[start:row_index])


def parse_languages_as_rows(lines: list[str]) -> dict[str, int | None] | None:
    values: dict[str, int | None] = {language: None for language in LANGUAGES}
    matched = False
    for i, line in enumerate(lines):
```

```
        stripped = line.strip()
        language = next(
            (name for name in LANGUAGES if re.match(rf"^{name}\b", stripped, re.IGNORECASE)),
            None,
        )
        if language is None:
            continue
        numbers = [parse_int(m.group(0)) for m in NUMBER.finditer(stripped)]
        if not numbers:
            continue
        if not _has_total_header(lines, i):
            raise ValueError(
                f"{language} row has numeric fields but no nearby Total header"
            )
        values[language] = numbers[-1]
        matched = True
    return values if matched else None


def _column_positions(header: str) -> dict[str, int]:
    # pdftotext -layout retains page spacing closely enough here for character
    # positions to identify the English and Hindi columns.
    positions = {}
    lower = header.lower()
    for language in LANGUAGES:
        pos = lower.find(language)
        if pos >= 0:
            positions[language] = pos
    return positions


def _nearest_number_at_column(line: str, position: int) -> int | None:
    matches = list(NUMBER.finditer(line))
    if not matches:
        return None
    # Match the language heading to the nearest numeric field on the Total row.
    nearest = min(matches, key=lambda m: abs(m.start() - position))
    return parse_int(nearest.group(0))


def parse_languages_as_columns(lines: list[str]) -> dict[str, int | None] | None:
```

```
    header_index = None
    positions: dict[str, int] = {}
    for i, line in enumerate(lines):
        candidate = _column_positions(line)
        if candidate:
            header_index = i
            positions = candidate
            break
    if header_index is None:
        return None

    total_line = next(
        (
            line
            for line in lines[header_index + 1 :]
            if re.match(r"^\s*(?:grand\s+)?total\b", line, re.IGNORECASE)
        ),
        None,
    )
    if total_line is None:
        raise ValueError("language columns found, but no Total or Grand Total row was found")

    return {
        language: _nearest_number_at_column(total_line, position)
        for language, position in positions.items()
    } | {language: None for language in LANGUAGES if language not in positions}


def parse_language_counts(text: str) -> dict[str, int | None]:
    lines = medium_block(text)
    parsed = parse_languages_as_rows(lines)
    if parsed is None:
        parsed = parse_languages_as_columns(lines)
    if parsed is None or parsed.get("english") is None:
        raise ValueError("could not resolve English enrollment from medium-of-instruction table")
    return parsed


def preferred_rows(rows: Iterable[dict[str, str]]) -> list[dict[str, str]]:
    grouped: dict[tuple[str, str, str], list[dict[str, str]]] = defaultdict(list)
    for row in rows:
```

```
        key = (
            row["academic_year"].strip(),
            row["state_report"].strip(),
            row["district_report"].strip(),
        )
        grouped[key].append(row)

    selected = []
    for key, group in grouped.items():
        priorities = [int(row["report_priority"]) for row in group]
        best = min(priorities)
        winners = [row for row in group if int(row["report_priority"]) == best]
        if len(winners) != 1:
            raise ValueError(f"multiple reports share the highest priority for {key}")
        selected.append(winners[0])
    return sorted(
        selected,
        key=lambda row: (
            row["academic_year"],
            row["state_report"],
            row["district_report"],
        ),
    )


def rebuild(
    metadata: Path,
    registry: Path,
    archive_root: Path,
    pdftotext: str,
) -> list[dict[str, str]]:
    report_map = registered_reports(registry)
    rebuilt = []
    for row in preferred_rows(read_csv(metadata)):
        source = row["source_pdf"].strip()
        if source not in report_map:
            raise ValueError(f"report metadata references an unregistered PDF: {source}")
        pdf = archive_root / report_map[source]
        if not pdf.is_file():
            raise FileNotFoundError(f"missing registered DISE report PDF: {pdf}")
        page = int(row["source_page"])
```

```
        counts = parse_language_counts(extract_pdf_page(pdf, page, pdftotext))
        out = dict(row)
        out["english_enrollment"] = str(counts["english"])
        out["hindi_enrollment"] = "" if counts["hindi"] is None else str(counts["hindi"])
        rebuilt.append(out)
    return rebuilt
```

Constructing the NSS Survey Design

In the 2007–08 National Sample Survey education round, households are sampled within first-stage units such as villages and urban blocks. For the enrollment probit, I define each sampling stratum from state, rural/urban sector, stratum, and substratum. I use the household multiplier as the sampling weight and the first-stage unit number as the cluster identifier. I pass these variables to `survey::svydesign()` with `nest = TRUE`, which relabels repeated cluster identifiers within strata. I fit the final survey-weighted probit only when all six required survey design fields are present.

File: `R/selection/estimate_selection_probit.R`

```
# List NSS fields needed to reproduce the survey sample design
#
selection_survey_design_variables <- function() {
  c("FSU_SL_NO", "weight", "STATE", "SECTOR", "STRATUM", "SUB_STRATUM_NO")
}

# Construct the NSS design used for weighted enrollment estimation
#
build_survey_design_selection <- function(selection_df, require_all = FALSE) {
  # Final estimation calls this with require_all = TRUE.
  required <- selection_survey_design_variables()
  missing <- setdiff(required, names(selection_df))
  if (isTRUE(require_all) && length(missing)) {
    stop(
      "NSS survey design is missing required fields: ",
      paste(missing, collapse = ", "),
      call. = FALSE
    )
  }
```

```
  psu <- first_col(selection_df, c("FSU_SL_NO", "fsu", "PSU", "psu"))
  weight <- first_col(selection_df, c("weight", "WEIGHT", "Multiplier", "multiplier"))
  strata_cols <- intersect(
    setdiff(required, c("FSU_SL_NO", "weight")),
    names(selection_df)
  )
  if (is.null(psu) || is.null(weight) || !length(strata_cols)) {
    if (isTRUE(require_all)) {
      stop("The required NSS survey design could not be constructed.", call. = FALSE)
    }
    return(NULL)
  }
  # NSS strata are defined by the joint state, rural/urban sector, stratum, and
  # substratum identifiers used in the survey files.
  selection_df$.survey_strata <- interaction(selection_df[strata_cols], drop = TRUE)
  # FSU identifiers can repeat across NSS strata; nest = TRUE relabels them to
  # enforce nesting within the joint sampling stratum.
  survey::svydesign(
    ids = stats::as.formula(paste0("~", psu)),
    strata = ~.survey_strata,
    weights = stats::as.formula(paste0("~", weight)),
    data = selection_df,
    nest = TRUE
  )
}
```

Average Marginal Effects on the Estimation Sample

For each fitted enrollment probit, I recover the observations actually used in estimation from its model frame and obtain the corresponding NSS sampling weights from the fitted survey design. I require one finite, nonnegative weight for each fitted observation and a positive total weight. I then pass those observations through `newdata` and the sampling weights through `wts` to `marginaleffects::avg_slopes()`. I use the model's survey-based covariance matrix for uncertainty estimates. For continuous covariates, I report average slopes on the predicted probability scale. For binary and categorical covariates, `marginaleffects` computes average discrete comparisons.

File: `R/selection/compute_average_marginal_effects.R`

```
# Recover observations and sampling weights used to fit the enrollment model
#
```

```r
ame_model_data_and_weights <- function(model) {
  model_data <- as.data.frame(stats::model.frame(model))

  model_weights <- if (inherits(model, "svyglm") && !is.null(model$survey.design)) {
    stats::weights(model$survey.design, type = "sampling")
  } else {
    tryCatch(stats::weights(model, type = "prior"), error = function(e) NULL)
  }

  if (is.null(model_weights)) model_weights <- rep(1, nrow(model_data))
  model_weights <- as.numeric(model_weights)
  if (length(model_weights) != nrow(model_data)) {
    stop("AME averaging weights must have one value per fitted observation.", call. = FALSE)
  }
  if (any(!is.finite(model_weights)) || any(model_weights < 0) || sum(model_weights) <= 0) {
    stop("AME averaging weights must be finite, nonnegative, and have positive total weight.", call. = FALSE)
  }

  list(data = model_data, wts = model_weights)
}

# Compute average marginal effects over the fitted NSS sample
#
compute_ames_marginaleffects <- function(model) {
  # Evaluate effects on the fitted model frame and use its sampling weights when
  # averaging observation level quantities on the response scale. For binary and
  # categorical regressors, marginaleffects reports discrete comparisons.
  amed <- ame_model_data_and_weights(model)
  marginaleffects::avg_slopes(
    model,
    newdata = amed$data,
    wts = amed$wts,
    vcov = TRUE,
    type = "response"
  )
}
```

Output: `outputs/tables/appendix/appendix_selection_ame.tex`

Rendering code: `R/output/appendix_selection_exhibits.R` and `R/output/save_tables.R`

Table 7: Average Marginal Effects for School Enrollment

Term	Enrolled (1 = yes)
Age (years)	-0.028***
	(0.000)
Female (ref: Male)	0.044***
	(0.003)
Household size	-0.004***
	(0.001)
Religion: Muslim (ref: Hindu)	-0.126***
	(0.007)
Religion: Christian	-0.043***
	(0.013)
Religion: Sikh	0.040
	(0.026)
Religion: Jain	-0.001
	(0.012)
Religion: Buddhist	0.047*
	(0.024)
Religion: Zoroastrian	0.101***
	(0.034)
Religion: Other	0.252***
	(0.003)
Social group: Scheduled Tribe (ref: Other)	-0.102***
	(0.008)
Social group: Scheduled Caste	-0.075***
	(0.006)
Social group: Other Backward Class	-0.029***
	(0.005)
Urban (ref: Rural)	-0.003
	(0.005)
Distance 1–2km (ref: <1km)	-0.077***
	(0.026)
Distance 2–3km	-0.001
	(0.010)
Distance 3–5km	0.004
	(0.066)
Distance > 5km	0.036
	(0.035)
Male household education: Literate, no school (ref: Illiterate)	0.097***
	(0.017)
Male household education: Literate, school < primary	0.127***
	(0.006)
Male household education: Primary	0.142***
	(0.005)
Male household education: Upper primary	0.195***
	(0.006)
Male household education: Secondary	0.212***
	(0.007)
Male household education: Higher secondary	0.247***
	(0.008)
Male household education: Postsecondary+	0.245***

	(0.009)

* $p < 0.10$, ** $p < 0.05$, *** $p < 0.01$. Entries are average changes in predicted enrollment probability. Continuous covariates are average slopes; categorical covariates are average discrete comparisons against the stated reference category, evaluated over the observed covariate distribution. Standard errors use the NSS sampling design, including household weights, primary sampling units, and strata formed from state, rural/urban sector, stratum, and substratum identifiers. The model also controls for district averages of schooling costs and benefits among enrolled children; the table displays marginal effects for child and household covariates. Source: NSS 64th Round, 2007–08, Participation and Expenditure in Education.

Identification and Econometric Robustness

Assessing Weak-Instrument Strength with the Montiel Olea–Pflueger Effective F

I use linguistic distance from Hindi as an instrument for district exposure to English-medium schooling. For each IV specification with one endogenous regressor, I recover the fitted regression formula, instrument formula, estimation observations, and state clusters from the `ivreg` fit. I then construct the corresponding `momentfit` model and call `MOPtest()` with a benchmark of 10% relative bias for TSLS, a 5% test size, and the simplified critical value. I report the resulting effective F and critical value as the instrument-strength results under the covariance assumptions used by the test. The accompanying table places these results beside the two-stage least squares estimates and Anderson–Rubin inference.

File: `R/iv/weak_identification.R`

```
mop_effective_f <- function(
    model, analysis_data, tau = 0.10, size = 0.05) {
  unavailable <- function(reason) {
    list(
      statistic = NA_real_, critical_value = NA_real_, p.value = NA_real_,
      effective_df = NA_real_, tau = tau, size = size,
      status = "not_estimated", reason = reason
    )
  }

  if (!inherits(model, "ivreg")) {
    return(unavailable("Montiel Olea-Pflueger effective F requires an ivreg model."))
  }
  if (!requireNamespace("momentfit", quietly = TRUE)) {
    return(unavailable("Package 'momentfit' is not installed."))
  }
  if (!is.finite(tau) || tau <= 0 || !is.finite(size) || size <= 0 || size >= 1) {
```

```
  return(unavailable("MOP tau must be positive, and size must lie strictly between 0 and 1."))
}

endogenous <- suppressWarnings(as.integer(model$endogenous))
if (length(endogenous) != 1L || !is.finite(endogenous)) {
  return(unavailable("Montiel Olea-Pflueger effective F requires exactly one endogenous regressor."))
}

regression_formula <- tryCatch(
  stats::formula(model, component = "regressors"),
  error = function(e) NULL
)
instrument_formula <- tryCatch(
  stats::formula(stats::delete.response(stats::terms(model, component = "instruments"))),
  error = function(e) NULL
)
if (is.null(regression_formula) || is.null(instrument_formula)) {
  return(unavailable("Could not recover the fitted regression and instrument formulas from ivreg."))
}
needed <- unique(c(all.vars(regression_formula), all.vars(instrument_formula)))
data <- iv_analysis_frame(analysis_data, needed)
missing <- setdiff(needed, names(data))
if (length(missing)) {
  return(unavailable(paste(
    "Missing variables for Montiel Olea-Pflueger effective F:",
    paste(missing, collapse = ", ")
  )))
}
moment_data <- data[needed]
# Keep MOP on the fitted IV sample; deleting additional rows for missing
# values would change the calculation.
if (any(!stats::complete.cases(moment_data))) {
  return(unavailable("Regressors or instruments used by the fitted IV model contain missing values."))
}

stored_cluster <- attr(model, "cluster_state", exact = TRUE)
cluster <- if (
  length(stored_cluster) == stats::nobs(model) &&
    !anyNA(stored_cluster) && length(unique(stored_cluster)) >= 2L
) {
  as.vector(stored_cluster)
```

```
} else {
  iv_model_cluster(model, data)
}
covariance <- if (is.null(cluster)) "MDS" else "CL"
covariance_options <- if (is.null(cluster)) {
  list(type = "HC0")
} else {
  # MOPtest() uses clustered HC0 moment covariance with its finite cluster
  # adjustment here. The separate first-stage Wald statistic uses CR2
  # covariance and the HTZ small sample reference.
  list(
    cluster = data.frame(cluster = cluster),
    type = "HC0", cadjust = TRUE, multi0 = FALSE
  )
}

result <- tryCatch({
  moment_model <- momentfit::momentModel(
    regression_formula,
    instrument_formula,
    data = moment_data,
    vcov = covariance,
    vcovOptions = covariance_options
  )
  # tau specifies the desired relative-bias benchmark for TSLS; the
  # simplified test uses the more conservative critical value.
  momentfit::MOPtest(
    moment_model,
    tau = tau,
    size = size,
    estMethod = "TSLS",
    simplified = TRUE,
    print = FALSE
  )
}, error = function(e) e)
if (inherits(result, "error")) return(unavailable(conditionMessage(result)))

values <- suppressWarnings(as.numeric(result[c("Feff", "critValue", "pvalue", "Keff")]))
names(values) <- c("statistic", "critical_value", "p.value", "effective_df")
if (length(values) != 4L || any(!is.finite(values))) {
  return(unavailable("momentfit::MOPtest() did not return finite MOP results."))
```

```
  }

  c(
    as.list(values),
    list(tau = tau, size = size, status = "estimated", reason = NA_character_)
  )
}
```

Output: `outputs/tables/appendix/appendix_iv_weak_inference.tex`

Rendering code: `R/output/appendix_identification_exhibits.R` and `R/output/save_tables.R`

Table 10: Inference with Weak Instruments for Long-Run Consumption Change

	Real log consumption change, 2004-05 to 2022-23	Real log consumption change, 2004-05 to 2023-24
EMI exposure	0.117	0.137
	(0.182)	(0.281)
MOP effective F	0.69	0.41
AR p-value at zero	0.029	0.081
AR 95% accepted set	$[-0.193, -0.030]^{\dagger} \cup [0.009, 0.193]^{\dagger}$	$[-0.186, 0.186]^{\dagger}$
Direct effect threshold for $\beta = 0$ (share of |reduced form|)	16.6%	0.0%
Observations	524	522

The table reports conventional 2SLS coefficients with state-clustered standard errors. Instrument strength is summarized by the Montiel Olea–Pflueger effective F statistic, and Anderson–Rubin tests provide inference robust to weak instruments. The 95% AR accepted sets are displayed as unions of closed intervals. †A component reaches an evaluated search boundary; the displayed outer endpoint is the numerical search limit. The direct effect threshold is the smallest permitted direct effect, expressed as a share of the absolute reduced form, for which the AR set includes $\beta = 0$. Sources: Census of India 2001; NSS 61st Round, 2004–05; NSS 64th Round, 2007–08; HCES 2022–23; HCES 2023–24.

Constructing Falsification-Adaptive Sets for Overidentified IV Designs

I use the falsification-adaptive set to assess how the estimated effect changes when the exclusion restrictions are relaxed one instrument at a time. For each candidate instrument, I estimate a just-identified two-stage least squares specification in which that instrument remains excluded and the other candidate instruments enter as controls. I use the same observations for every constituent estimate by requiring complete data on all variables in the full specification, and I cluster inference by state. The lower and upper endpoints of the FAS are the smallest and largest constituent estimates. I also report each instrument's conditional first-stage statistic as evidence about the maintained relevance assumption.

File: `R/iv/falsification_adaptive_set.R`

```
# For the exclusion relaxation used here, each relevant instrument serves in
# turn as the sole excluded instrument while the remaining instruments enter as
# controls. The FAS spans the resulting just-identified IV estimates. Every
# registered constituent enters the set calculation; conditional first-stage
# statistics separately summarize evidence on instrument relevance.
iv_fas_component_formula <- function(specification, instrument) {
  spec <- as_single_iv_specification(specification)
  excluded <- plain_chr(unlist(spec$excluded_instruments[[1L]], use.names = FALSE))
  instrument <- plain_chr(instrument)[[1L]]
  if (!instrument %in% excluded) {
    stop("Selected FAS instrument is absent from the specification's excluded instruments.", call. = FALSE)
```

```
  }
  included <- unique(c(
    unlist(spec$included_language_controls[[1L]], use.names = FALSE),
    unlist(spec$controls[[1L]], use.names = FALSE),
    setdiff(excluded, instrument)
  ))
  make_iv_formula(
    spec$outcome[[1L]], spec$treatment[[1L]], instrument,
    controls = included,
    fixed_effects = iv_fixed_effect_terms(spec$fixed_effect[[1L]])
  )
}

estimate_iv_fas_component <- function(data, specification, instrument) {
  spec <- as_single_iv_specification(specification)
  excluded <- plain_chr(unlist(spec$excluded_instruments[[1L]], use.names = FALSE))
  instrument <- plain_chr(instrument)[[1L]]
  other_instruments <- setdiff(excluded, instrument)
  # Resolve transformed formula terms such as factor(region) to the underlying
  # columns used to define the common estimation sample.
  needed <- iv_specification_variables(spec)
  missing <- setdiff(needed, names(data))
  if (length(missing)) {
    stop(
      "FAS specification ", spec$specification_id[[1L]],
      " is missing columns: ", paste(missing, collapse = ", "),
      call. = FALSE
    )
  }
  x <- as.data.frame(data)
  x <- x[stats::complete.cases(x[needed]), , drop = FALSE]
  if (nrow(x) < 3L) {
    return(data.frame(
      specification_id = spec$specification_id[[1L]],
      instrument = instrument, estimate = NA_real_, std.error = NA_real_,
      p.value = NA_real_, first_stage_estimate = NA_real_,
      first_stage_std.error = NA_real_, first_stage_f = NA_real_,
      first_stage_p.value = NA_real_, n = nrow(x), status = "not_estimated",
      reason = "Insufficient complete observations.", stringsAsFactors = FALSE
    ))
  }
```

```
formula <- iv_fas_component_formula(spec, instrument)
fit <- tryCatch(
  ivreg::ivreg(formula, data = x, model = TRUE, x = TRUE, y = TRUE),
  error = function(e) e
)
if (inherits(fit, "error")) {
  return(data.frame(
    specification_id = spec$specification_id[[1L]],
    instrument = instrument, estimate = NA_real_, std.error = NA_real_,
    p.value = NA_real_, first_stage_estimate = NA_real_,
    first_stage_std.error = NA_real_, first_stage_f = NA_real_,
    first_stage_p.value = NA_real_, n = nrow(x), status = "not_estimated",
    reason = conditionMessage(fit), stringsAsFactors = FALSE
  ))
}

cluster <- iv_specification_cluster(x, spec)
inference <- iv_clustered_inference(fit, cluster)
treatment_term <- model_term_inference(
  fit, spec$treatment[[1L]], inference$vcov
)

included <- unique(c(
  unlist(spec$included_language_controls[[1L]], use.names = FALSE),
  unlist(spec$controls[[1L]], use.names = FALSE),
  other_instruments,
  iv_fixed_effect_terms(spec$fixed_effect[[1L]])
))
first_stage <- stats::lm(
  stats::reformulate(unique(c(instrument, included)), response = spec$treatment[[1L]]),
  data = x
)
first_stage_inference <- iv_clustered_inference(first_stage, cluster)
first_stage_term <- model_term_inference(
  first_stage, instrument, first_stage_inference$vcov
)
first_stage_joint <- clustered_joint_wald_test(
  first_stage, instrument, cluster, inference = first_stage_inference
)
first_stage_f <- unname(first_stage_joint[["statistic"]])
```

```
  treatment_ok <- all(is.finite(
    treatment_term[c("estimate", "std.error", "p.value")]
  ))
  first_stage_ok <- all(is.finite(
    first_stage_term[c("estimate", "std.error", "p.value")]
  ))
  status <- if (
    identical(inference$status, "estimated") &&
      identical(first_stage_inference$status, "estimated") &&
      treatment_ok && first_stage_ok
  ) "estimated" else "inference_unavailable"
  reasons <- unique(na.omit(c(
    if (identical(inference$status, "unavailable")) inference$reason else NA_character_,
    if (identical(first_stage_inference$status, "unavailable")) first_stage_inference$reason else
     ↪  NA_character_,
    if (identical(inference$status, "estimated") && !treatment_ok)
      "Treatment coefficient inference is nonfinite." else NA_character_,
    if (identical(first_stage_inference$status, "estimated") && !first_stage_ok)
      "Conditional first-stage coefficient inference is nonfinite." else NA_character_
  )))

  data.frame(
    specification_id = spec$specification_id[[1L]],
    instrument = instrument,
    estimate = treatment_term[["estimate"]],
    std.error = treatment_term[["std.error"]],
    p.value = treatment_term[["p.value"]],
    first_stage_estimate = first_stage_term[["estimate"]],
    first_stage_std.error = first_stage_term[["std.error"]],
    first_stage_f = first_stage_f,
    first_stage_p.value = unname(first_stage_joint[["p.value"]]),
    n = stats::nobs(fit),
    status = status,
    reason = if (length(reasons)) paste(reasons, collapse = "; ") else NA_character_,
    stringsAsFactors = FALSE
  )
}

estimate_iv_falsification_adaptive_set_spec <- function(data, specification) {
  spec <- as_single_iv_specification(specification)
```

```
  excluded <- plain_chr(unlist(spec$excluded_instruments[[1L]], use.names = FALSE))
  if (spec$n_endogenous[[1L]] != 1L || length(excluded) <= 1L) {
    stop("FAS requires one endogenous regressor and multiple excluded instruments.", call. = FALSE)
  }
  components <- safe_bind_rows(lapply(excluded, function(instrument) {
    estimate_iv_fas_component(data, spec, instrument)
  }))
  estimated <- components$status == "estimated" & is.finite(components$estimate)
  complete <- length(estimated) == length(excluded) && all(estimated)

  lower <- if (complete) min(components$estimate) else NA_real_
  upper <- if (complete) max(components$estimate) else NA_real_
  lower_instrument <- if (complete) components$instrument[which.min(components$estimate)] else NA_character_
  upper_instrument <- if (complete) components$instrument[which.max(components$estimate)] else NA_character_
  first_stage_f <- num(components$first_stage_f)

  summary <- data.frame(
    specification_id = spec$specification_id[[1L]],
    adjustment_id = spec$adjustment_id[[1L]],
    construction_id = spec$construction_id[[1L]],
    n_instruments = length(excluded),
    n_components_estimated = sum(estimated),
    fas_lower = lower,
    fas_upper = upper,
    fas_width = if (complete) upper - lower else NA_real_,
    fas_contains_zero = if (complete) lower <= 0 && upper >= 0 else NA,
    lower_endpoint_instrument = lower_instrument,
    upper_endpoint_instrument = upper_instrument,
    min_conditional_first_stage_f = if (all(is.finite(first_stage_f))) min(first_stage_f) else NA_real_,
    max_conditional_first_stage_f = if (all(is.finite(first_stage_f))) max(first_stage_f) else NA_real_,
    n_conditional_first_stage_f_below_10 = sum(is.finite(first_stage_f) & first_stage_f < 10),
    any_conditional_first_stage_f_below_10 = if (all(is.finite(first_stage_f))) any(first_stage_f < 10) else
      ↪ NA,
    n = if (nrow(components) && length(unique(components$n)) == 1L) components$n[[1L]] else NA_integer_,
    status = if (complete) "estimated" else "incomplete",
    reason = if (complete) NA_character_ else "At least one registered just-identified constituent could not
      ↪ be estimated.",
    stringsAsFactors = FALSE
  )
  list(summary = summary, components = components)
}
```

Price Harmonization and Real Consumption

Linking Historical and Later State CPI Series onto a Common Scale

For rural areas, I link the historical CPI-RL to the later state rural CPI. For urban areas, I link the state CPI-IW to the later state urban CPI. Within each state and rural/urban sector, I calculate the ratio of the later CPI to the historical CPI for every paired month from January 2013 through December 2014 and use the median ratio as the link factor. I require at least six paired months for every historical series used before January 2013. I also retain the first and last monthly ratios, the median absolute deviation, the minimum and maximum ratios, and the number of paired months. I multiply the historical observations by the link factor before January 2013 and use the later CPI directly from January 2013 onward.

File: `R/prices/build_temporal_price_series.R`

```
# Link an earlier CPI series to a later one with the median ratio across valid
# overlap months.
price_link_factor <- function(old_index, new_index) {
  old <- num(old_index)
  new <- num(new_index)
  keep <- positive_finite(old) & positive_finite(new)
  if (!any(keep)) return(NA_real_)
  stats::median(new[keep] / old[keep], na.rm = TRUE)
}


build_temporal_price_series <- function(
    price_sources,
    switch_date = as.Date("2013-01-01"),
    overlap_start = as.Date("2013-01-01"),
    overlap_end = as.Date("2014-12-01"),
    minimum_link_months = 6L,
    pre_switch_start = NULL,
    pre_switch_end = NULL) {
  switch_date <- price_boundary(switch_date)
  old <- select_pre_2013_price_series(price_sources)
  new <- select_post_2013_price_series(price_sources)
  links <- summarise_price_links(old, new, overlap_start, overlap_end)

  minimum_link_months <- as.integer(minimum_link_months)
  if (length(minimum_link_months) != 1L || is.na(minimum_link_months) || minimum_link_months < 1L) {
    stop("minimum_link_months must be one positive integer.", call. = FALSE)
```

```
}

old_groups <- unique(old[old$period < switch_date, c("state_code", "sector"), drop = FALSE])
linked_groups <- links[links$link_months >= minimum_link_months, c("state_code", "sector"), drop = FALSE]
linked_key <- paste(linked_groups$state_code, linked_groups$sector, sep = "\r")
missing_key <- !paste(old_groups$state_code, old_groups$sector, sep = "\r") %in% linked_key
if (any(missing_key)) {
  bad <- old_groups[missing_key, , drop = FALSE]
  stop(
    "Pre-2013 price series have fewer than the required overlap months with CPI-R/U: ",
    paste(paste(bad$state_code, bad$sector, sep = "/"), collapse = ", "),
    call. = FALSE
  )
}

old <- merge(old, links, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
old <- old[old$period < switch_date, , drop = FALSE]
if (!is.null(pre_switch_start) || !is.null(pre_switch_end)) {
  if (is.null(pre_switch_start) || is.null(pre_switch_end)) {
    stop("Both pre_switch_start and pre_switch_end are required when trimming the old price series.",
     ↪  call. = FALSE)
  }
  pre_switch_start <- price_boundary(pre_switch_start)
  pre_switch_end <- price_boundary(pre_switch_end)
  if (pre_switch_end < pre_switch_start || pre_switch_end >= switch_date) {
    stop("The retained historical interval must be ordered and end before the CPI switch date.", call. =
     ↪  FALSE)
  }
  old <- old[old$period >= pre_switch_start & old$period <= pre_switch_end, , drop = FALSE]
}
old$index_unlinked <- old$index
# Put each historical CPI series on the CPI-R/U scale before appending
# observations from January 2013 onward.
old$index <- num(old$index) * num(old$link_factor)

new <- new[new$period >= switch_date, , drop = FALSE]
new$link_factor <- 1
new$link_months <- NA_integer_
new$link_ratio_mad <- NA_real_
new$link_ratio_min <- NA_real_
new$link_ratio_max <- NA_real_
```

```
  new$index_unlinked <- new$index

  columns <- c(
    "state_code", "sector", "period", "index", "index_unlinked", "price_source",
    "link_factor", "link_months", "link_ratio_mad", "link_ratio_min", "link_ratio_max"
  )
  index <- rbind(old[columns], new[columns])
  index$year <- as.integer(format(index$period, "%Y"))
  index$month <- as.integer(format(index$period, "%m"))
  index <- index[order(index$state_code, index$sector, index$period), , drop = FALSE]
  rownames(index) <- NULL
  validate_temporal_price_chain(index, switch_date)

  structure(
    list(index = index, links = links, switch_date = switch_date),
    class = "emi_temporal_price_series"
  )
}
```

Constructing Comparable Rural and Urban Price Deflators Across States

I express household consumption on a common 2011–12 price scale in two steps. First, I divide each state's rural or urban Tendulkar poverty line by 816, the all-India rural Tendulkar line for 2011–12. I use this ratio to account for differences in price levels across states and between rural and urban areas. Second, I divide each month's CPI by the corresponding 2011–12 reference CPI to measure price change over time. I multiply these two ratios to obtain the monthly deflator. At the reference period, an amount equal to the applicable Tendulkar poverty line therefore equals 816 on the common real consumption scale. When a state uses a declared donor CPI series, I use the donor state's reference CPI in the denominator as well. I then compare district real consumption estimates from the 2022–23 and 2023–24 Household Consumption Expenditure Survey rounds in the accompanying figure.

File: `R/prices/price_deflators.R`

```
build_state_sector_deflator <- function(
    temporal_index, spatial_relatives, reference_period = NULL, reference_index = NULL) {
  idx <- safe_df(temporal_index)
  spatial <- safe_df(spatial_relatives)
  validate_price_index(idx, keys = c("state_code", "sector", "period"))
  required_spatial <- c("state_code", "sector", "spatial_price_relative")
  missing <- setdiff(required_spatial, names(spatial))
```

```r
if (length(missing)) {
  stop("Spatial price relatives are missing columns: ", paste(missing, collapse = ", "), call. = FALSE)
}
if (anyDuplicated(spatial[c("state_code", "sector")])) {
  stop("Spatial price relatives must have one row per state and sector.", call. = FALSE)
}
if (any(!positive_finite(num(spatial$spatial_price_relative)))) {
  stop("Spatial price relatives must be positive and finite.", call. = FALSE)
}

if (is.null(reference_index)) {
  if (is.null(reference_period)) stop("A price reference period or reference-index table is required.",
   ↪  call. = FALSE)
  reference_period <- as.Date(reference_period)
  ref <- idx[idx$period %in% reference_period, , drop = FALSE]
  if (!nrow(ref)) stop("No observations fall in the requested price reference period.", call. = FALSE)
  ref_mean <- stats::aggregate(index ~ state_code + sector, ref, function(x) mean(num(x), na.rm = TRUE))
  names(ref_mean)[names(ref_mean) == "index"] <- "reference_index"
} else {
  ref_mean <- safe_df(reference_index)
  required_reference <- c("state_code", "sector", "reference_index")
  missing_reference <- setdiff(required_reference, names(ref_mean))
  if (length(missing_reference)) {
    stop("Reference price table is missing columns: ", paste(missing_reference, collapse = ", "), call. =
     ↪  FALSE)
  }
  if (anyDuplicated(ref_mean[c("state_code", "sector")]) ||
   ↪  any(!positive_finite(ref_mean$reference_index))) {
    stop("Reference price table must have one positive row per state and rural/urban sector.", call. =
     ↪  FALSE)
  }
  ref_mean <- ref_mean[required_reference]
}
out <- merge(
  idx,
  spatial,
  by = c("state_code", "sector"), all.x = TRUE, sort = FALSE
)
out <- merge(out, ref_mean, by = c("state_code", "sector"), all.x = TRUE, sort = FALSE)
if ("source_state_code" %in% names(out) && any(!positive_finite(out$reference_index))) {
  # When a state and sector use a declared donor CPI series, use the donor's
```

```
    # reference period index as well so both terms come from the same CPI
    # series.
    donor_reference <- ref_mean
    names(donor_reference)[names(donor_reference) == "state_code"] <- "source_state_code"
    names(donor_reference)[names(donor_reference) == "reference_index"] <- "donor_reference_index"
    out <- merge(out, donor_reference, by = c("source_state_code", "sector"), all.x = TRUE, sort = FALSE)
    missing_reference <- !positive_finite(out$reference_index)
    out$reference_index[missing_reference] <- out$donor_reference_index[missing_reference]
    out$donor_reference_index <- NULL
  }
  # The product adjusts for the rural/urban price level in each state and for
  # price change from the reference period to the survey month.
  out$temporal_price_relative <- num(out$index) / num(out$reference_index)
  out$price_deflator <- num(out$spatial_price_relative) * num(out$temporal_price_relative)
  if (any(!positive_finite(out$price_deflator))) {
    stop("Price deflators must be positive and finite for every retained state, sector, and month.", call. =
      ↪ FALSE)
  }
  out <- out[order(out$state_code, out$sector, out$period), , drop = FALSE]
  rownames(out) <- NULL
  out
}
```

Output: `outputs/figures/appendix/appendix_consumption_hces_consistency.pdf`

Rendering code: `R/output/appendix_data_construction_exhibits.R` and `R/output/save_figures.R`

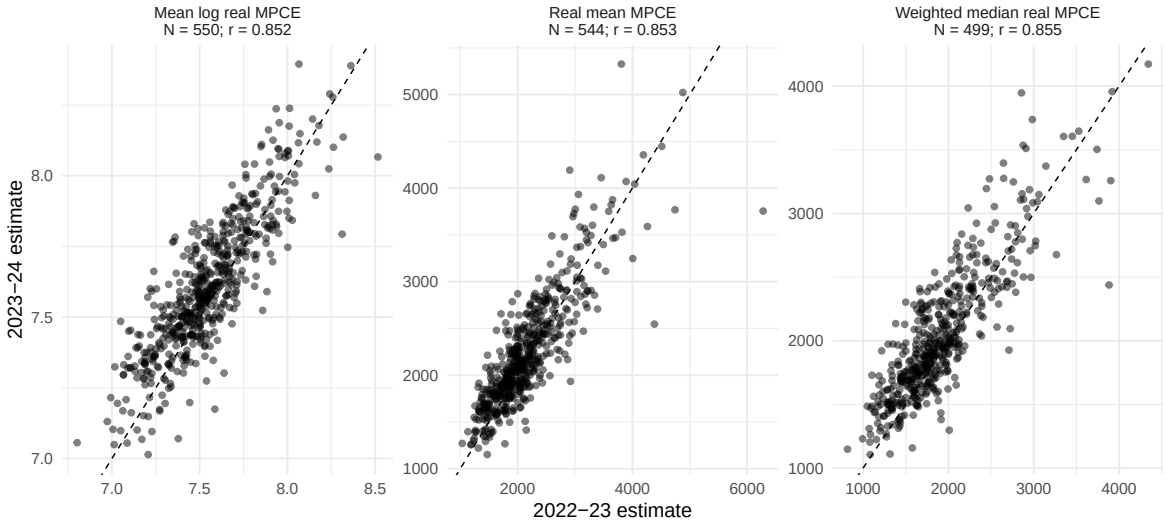


Figure 8: Agreement between district real-consumption estimates in the 2022–23 and 2023–24 HCES rounds. Each panel uses districts retained in both rounds under the consumption sample criteria; the dashed line is the 45-degree line. Sources: National Sample Survey Office, HCES 2022–23 and HCES 2023–24 (National Sample Survey Office 2024, 2025).

Testing CPI Links Against Known Answers

I test the CPI linking rule with values whose correct answer can be checked by hand. In the rural example, the monthly ratios of the later CPI to the historical CPI are 2, 2, and 3, so the median link is 2 even though the final monthly ratio is 3. In the urban example, all three ratios equal 0.5. I check the median link, the first and last ratios, and the number of paired months.

File: `tests/testthat/test-prices-and-controls.R`

```
test_that("CPI link factors equal median overlap ratios", {
  months <- as.Date(c("2013-01-01", "2013-02-01", "2013-03-01"))
  # Rural monthly ratios are 2, 2, and 3, so the median link is 2.
  old <- rbind(
    data.frame(state_code = "A", sector = "rural", period = months, index = c(100, 110, 120)),
    data.frame(state_code = "A", sector = "urban", period = months, index = c(200, 220, 240))
  )
  new <- rbind(
    data.frame(state_code = "A", sector = "rural", period = months, index = c(200, 220, 360)),
    data.frame(state_code = "A", sector = "urban", period = months, index = c(100, 110, 120))
  )
  links <- summarise_price_links(old, new, months[1], months[3])
  expect_equal(links$link_factor, c(2, 0.5))
```

```
  expect_equal(links$link_factor_first, c(2, 0.5))
  expect_equal(links$link_factor_last, c(3, 0.5))
  expect_equal(links$link_months, c(3L, 3L))
})
```

Spatial Presentation on a Common Geography

Preserving All 2001 Census Districts in Published Maps

For each published district map, I begin with the full reviewed 2001 Census district geometry and join the analysis values by district identifier. Every 2001 Census district therefore remains in the figure; districts without an estimate appear in the explicit "No data" category. I derive each state code from the 2001 Census district identifier and compare it with any state code supplied with the analysis values. I then use the completed district table to draw the geographic figure.

File: `R/output/save_figures.R`

```
complete_public_map_geometry <- function(district_panel, map_geometry) {
  if (!has_sf_geometry(map_geometry)) {
    stop("District maps require the complete 2001 Census district geometry.", call. = FALSE)
  }
  key <- "target_unit_2001"
  if (!key %in% names(map_geometry) && "unit_id" %in% names(map_geometry)) {
    map_geometry[[key]] <- map_geometry$unit_id
  }
  if (!key %in% names(map_geometry) || !key %in% names(district_panel)) {
    stop("Map geometry and analysis data must contain a 2001 Census district key.", call. = FALSE)
  }
  if (anyDuplicated(map_geometry[[key]])) {
    stop("Complete 2001 Census map geometry must contain one row per district.", call. = FALSE)
  }
  attributes <- if (inherits(district_panel, "sf")) sf::st_drop_geometry(district_panel) else
↪   safe_df(district_panel)
  if (anyDuplicated(attributes[[key]])) {
    stop("Analysis data must contain at most one row per 2001 Census district.", call. = FALSE)
  }
  out <- merge(map_geometry, attributes, by = key, all.x = TRUE, sort = FALSE)

  # Derive state codes from the 2001 Census district identifiers so state
```

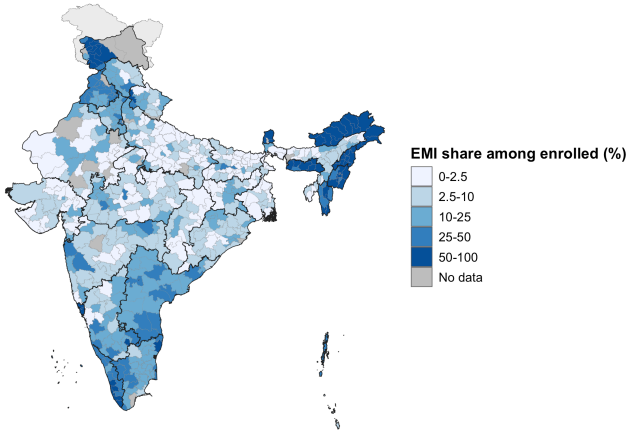
```
  # outlines include districts with missing analysis values.
  boundary_state <- public_map_state_code_2001(out[[key]])
  if ("state_code_2001" %in% names(out)) {
    panel_state <- plain_chr(out$state_code_2001)
    mismatch <- !is.na(panel_state) & nzchar(panel_state) & panel_state != boundary_state
    if (any(mismatch)) {
      stop("Analysis state codes disagree with the state codes in the 2001 Census district identifiers.",
       ↪  call. = FALSE)
    }
  }
  out$state_code_2001 <- boundary_state
  out
}

save_map_figure <- function(spec, path_base, district_panel, map_geometry, boundary_reference, formats) {
  if (is.null(spec$variable) || !spec$variable %in% names(district_panel)) {
    stop("Map figure '", spec$name, "' is missing variable '", spec$variable, "'.", call. = FALSE)
  }

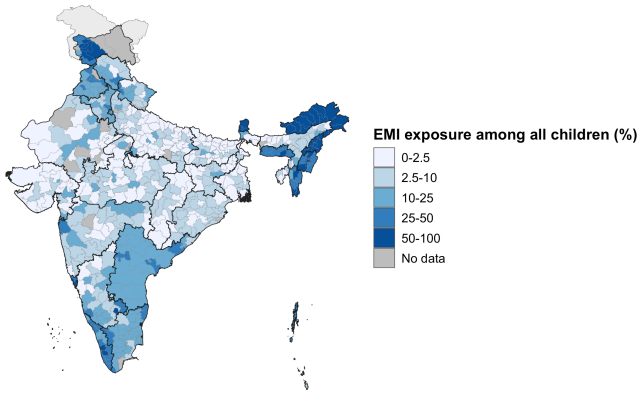
  plot_data <- complete_public_map_geometry(district_panel, map_geometry)
  plot_data <- prepare_public_map_data(plot_data, spec$variable)
  p <- build_public_ggplot_map(plot_data, spec, boundary_reference = boundary_reference)
  save_plot_formats(p, path_base, formats, width = 7.2, height = 5.2, dpi = 300)
}
```

Outputs: `outputs/figures/main/map_emi_share_enrolled.png`, `outputs/figures/main/map_emi_exposure.png`, `outputs/figures/main/map_public_emi_exposure.png`, and `outputs/figures/main/map_private_emi_exposure.png`

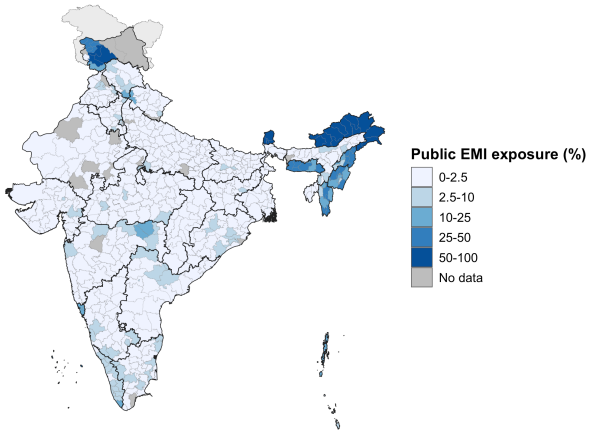
Rendering code: `R/output/make_figures.R`



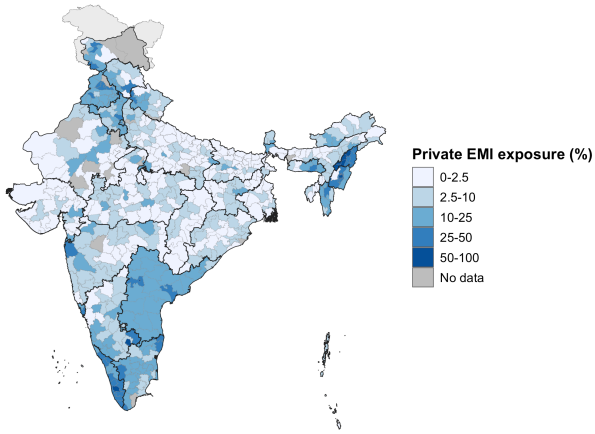
(a) EMI share among enrolled children.



(b) EMI exposure among all children.



(c) Public-school EMI exposure among all children.



(d) Private-school EMI exposure among all children.

Figure 3: District distribution of English-medium schooling measures on 2001 Census boundaries. All panels use a common percentage scale. Source: NSS 64th Round, 2007–08 (National Sample Survey Office 2011). Tengshe (2014) and Natural Earth (2022) provide geospatial data. District boundaries reflect 2001 geography. Politically disputed regions are identified using Natural Earth, and those without data—Aksai Chin, Azad Kashmir, Gilgit-Baltistan, the Trans-Karakoram Tract, and Siachen Glacier—are colored light gray. Their inclusion does not imply a position on territorial disputes.